

.

1. Loading Data

Command	What it does
<code>pd.read_csv('file.csv')</code>	Loads a CSV file into a DataFrame. (Swap out <code>_csv</code> for <code>_excel</code> , <code>_json</code> , or <code>_sql</code> depending on your source).
<code>pd.DataFrame(data)</code>	Creates a brand new DataFrame from scratch using a dictionary or a list of lists.

2. Inspecting Your Data

Command	What it does
<code>df.head(n)</code>	Previews the first <code>n</code> rows (defaults to 5). Great for a quick sanity check.
<code>df.info()</code>	Shows the structural overview: column names, data types, and how many missing (null) values you have.
<code>df.describe()</code>	Generates quick summary statistics (mean, median, min, max, etc.) for all numerical columns.
<code>df.shape</code>	Returns a tuple showing the dimensions of your

Command

What it does

`df['column'].value_counts()` Counts the unique values in a specific column.

3. Selecting & Filtering

Command	What it does
<code>df[['col1', 'col2']]</code>	Selects specific columns to work with or view.
<code>df.loc[row_label, col_label]</code>	Selects rows and columns by their text labels or names.
<code>df.iloc[row_index, col_index]</code>	Selects rows and columns by their integer positions (e.g., row 0 to 5).
<code>df[df['col'] > 50]</code>	Filters rows based on a condition (boolean indexing).

4. Cleaning Data

Command	What it does
<code>df.isnull().sum()</code>	Returns a count of how many missing values exist in each column.
<code>df = df.dropna(subset=['column_A', 'column_B'])</code>	Drops rows or columns that contain missing values.
<code>df.fillna(value)</code>	Replaces missing values with a specific number, string, or statistical value (like the mean).
<code>df.rename(columns={'old':'new'})</code>	Renames columns using a dictionary mapping.

5. Transforming & Aggregating

Command	What it does
<code>df['col'] = df['col'].apply(func)</code>	Applies a function (like a lambda or a custom Python function) to every element in a column.
<code>df.groupby('col').mean()</code>	Groups data by a categorical column and calculates an aggregate statistic (like mean or sum) for each group.
<code>df2= df.sort_values(by=['colA', 'colB'], ascending=[True, False])</code>	Sorts the rows based on the values of a specific column.
<code>pd.merge(df1, df2, on='colC')</code>	Joins two DataFrames together on a common column (similar to a SQL JOIN).
<code>df = pd.concat([df1, df2], ignore_index=True)</code>	Concat dfs vertically (stack on top)
<code>df2 = df[df['Age'] > 30].copy()</code>	<code>Copy()</code> used so that <code>SetCopy</code> warning does not come on <code>df2</code> operations after this
<code>For r in df.iterrows(): ...</code>	Iter over <code>df</code>

<i>Command</i>	<i>What it does</i>
<code>df = df.reset_index(drop=True)</code>	Cleans your index without adding a junk column
<code>df.set_index('colA', inplace=True)</code>	<i>Set index</i>
<pre>def f1(p, tax1): return p * (1 + tax) df['Total_Price'] = f1(df['Price'], df['Tax'])</pre>	<i>Faster than using lambda functions. Because here all rows are processed parallely</i>
<code>Dict1 = df.to_json(orient='records', indent=4)</code>	<i>With orient=records, the df columns will become keys in resulting list of dicts</i>
<pre>df['dtCol'] = pd.to_datetime(df['date'], format='%d/%m/%Y')</pre>	
<pre>df['date_str'] = df['date1'].dt.strftime('%Y-%m-%d')</pre>	
<code>df['colA'] = df['colA'].astype(int)</code>	